RNN Theory

Structured Notes

Sequential Data Recap, Many-to-One

and Many-to-Many RNN,
Training from Scratch, and PyTorch Implementation

Contents

1 Quick Recall: Sequential Data and RNN Basics **2**
1.1 Hidden State Idea . . . 2
1.2 Basic RNN Update Equation . . . 2
1.3 Hidden State Evolution Example . . . 2

2 Two Common RNN Output Styles **3**

3 Many-to-One RNN: Practical Implementation **3**
3.1 Toy Example Setup . . . 4
3.2 Word-to-Vector Conversion . . . 4
3.3 Model Parameters . . . 4
3.4 Forward Pass Function . . . 4
3.5 Key Observations from Many-to-One . . . 5

4 Many-to-Many RNN: Practical Implementation **5**
4.1 Toy Example: Next-Character Prediction . . . 6
4.2 Softmax Function . . . 6
4.3 Key Observations from Many-to-Many . . . 7

5 Comparison: Many-to-One vs Many-to-Many **7**

6 Training RNN from Scratch **7**
6.1 Training Components . . . 7
6.2 Training Flow . . . 8
6.3 Training Implementation (NumPy) . . . 8
6.4 What to Observe During Training . . . 9
6.5 Training Concepts . . . 9

7 Training Real RNN Using PyTorch **10**
7.1 Many-to-One with PyTorch . . . 10
7.2 PyTorch Model Architecture Breakdown . . . 11
7.3 Training Flow with PyTorch . . . 11

8 RNN Padding and Variable-Length Sequences **11**

9 Backpropagation Through Time (BPTT) **11**

10 Summary **12**

1 Quick Recall: Sequential Data and RNN Basics

Sequential Data

In **sequential data**, order matters. Examples include:

- Sentence: "I am happy"
- Time-series: sensor readings over time
- Stock prices over days
- Audio signals
- Character sequences

Unlike standard feedforward networks, sequence models must process information in order.

1.1 Hidden State Idea

RNN introduces a **hidden state** that acts like a running memory. At each time step:

1. The model reads the current input
2. Combines it with past memory
3. Updates its new memory

The hidden state carries information from previous time steps.

1.2 Basic RNN Update Equation

At time step t:

$$h_t = \tanh(W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1} + b_h)$$

where:

- x_t = current input
- h_{t-1} = previous hidden state
- h_t = new hidden state
- W_{xh} = weight matrix for input
- W_{hh} = weight matrix for hidden state
- b_h = bias term

If output is needed:

$$y_t = W_{hy} \cdot h_t + b_y$$

This means:

- Input flows through time
- Memory gets updated repeatedly
- Output can be taken at the final step or at every step

1.3 Hidden State Evolution Example

Hidden State Evolution

```python
import numpy as np
np.set_printoptions(precision=3, suppress=True)

# Sequence length = 4, input dimension = 2, hidden dimension = 3
x_seq = [
    np.array([[1.0], [0.0]]),
    np.array([[0.5], [1.0]]),
    np.array([[0.0], [1.0]]),
    np.array([[1.0], [1.0]])
```

```
]

W_xh = np.array([
    [0.5, -0.2],
    [0.3,  0.8],
    [-0.6, 0.1]
])

W_hh = np.array([
    [0.4, 0.1, -0.2],
    [0.0, 0.5,  0.3],
    [0.2, -0.4, 0.6]
])

b_h = np.array([[0.0], [0.0], [0.0]])
h = np.zeros((3, 1))

for t, x_t in enumerate(x_seq, start=1):
    h = np.tanh(W_xh @ x_t + W_hh @ h + b_h)
    print(f"Time step {t}, Hidden state: {h.T}")
```

Listing 1: Observing hidden state updates through a sequence

After every input, the hidden state is updated. This is how an RNN hidden state works — at each time step the model reads the current input, combines it with the past memory (previous hidden state), and produces an updated hidden state.

2 Two Common RNN Output Styles

Style	Description	Examples
Many-to-One	Whole sequence → one output	Sentiment classification, sequence classification
Many-to-Many	Sequence → output at each step	Next character prediction, POS tagging
One-to-Many	One input → sequence output	Image captioning

Many-to-One vs Many-to-Many

- **Many-to-One:** Loss is calculated only at the final step. The final hidden state contains cumulative information from all time steps.
- **Many-to-Many:** Loss is calculated at every time step. Output is generated at each step, and the training process differs — weights are updated based on losses at every step.

3 Many-to-One RNN: Practical Implementation

Many-to-One RNN

The entire input sequence is processed step-by-step, and only a **single output** is produced from the final hidden state.

$$\text{Sequence} \rightarrow \text{Hidden states} \rightarrow \text{Final hidden state} \rightarrow \text{Output}$$

After the full sequence is processed, the final hidden state becomes the representation of the entire sequence:

$$y = W_{hy} \cdot h_T + b_y$$

3.1 Toy Example Setup

A simple vocabulary is created:

- `good` → positive
- `bad` → negative

Sequences and labels (supervised learning):

- `[good, good, good, good]` → 1 (positive)
- `[bad, bad, bad, bad]` → 0 (negative)
- `[good, bad, good]` → 1 (positive)
- `[bad, good, bad]` → 0 (negative)

3.2 Word-to-Vector Conversion

Words are converted to numerical vectors (a form of one-hot encoding done manually):

Word-to-Vector Mapping

```
word_to_vec = {
    "good": np.array([[1.0], [0.0]]),
    "bad":  np.array([[0.0], [1.0]])
}
```

Listing 2: Manual word-to-vector encoding

3.3 Model Parameters

- **Input size** = 2 (each word is a 2-dimensional vector)
- **Hidden size** = 4 (4 hidden neurons, meaning 4 weights and 4 biases)
- **Output size** = 1 (binary classification: positive or negative)

Weight matrices are initialized randomly:

- W_{xh}: shape $(h, d) = (4, 2)$ — weight matrix for input
- W_{hh}: shape $(h, h) = (4, 4)$ — weight matrix for hidden state
- W_{hy}: shape $(o, h) = (1, 4)$ — weight matrix for output

Bias vectors are initialized to zeros.

Why Random Weight Initialization?

If weights are initialized to zero, then gradients will also be zero ($\frac{dW}{dT} = 0$), and the model will never update. Weights must be random so that gradient descent can begin learning. If all initial weights are zero, the vanishing gradient problem occurs immediately in the first pass.

3.4 Forward Pass Function

Many-to-One Forward Pass

```
import numpy as np
np.random.seed(42)

input_size = 2
hidden_size = 4
output_size = 1

W_xh = np.random.randn(hidden_size, input_size) * 0.5
```

```
W_hh = np.random.randn(hidden_size, hidden_size) * 0.5
W_hy = np.random.randn(output_size, hidden_size) * 0.5

b_h = np.zeros((hidden_size, 1))
b_y = np.zeros((output_size, 1))

def forward(sequence):
    h = np.zeros((hidden_size, 1))
    hidden_states = []

    for word in sequence:
        x = word_to_vec[word]
        h = np.tanh(W_xh @ x + W_hh @ h + b_h)
        hidden_states.append(h)

    y = W_hy @ h + b_y
    return y, hidden_states
```

Listing 3: Forward pass through the RNN

The `@` symbol represents matrix multiplication (dot product). The function:

1. Initializes hidden state h to zeros
2. Iterates through each word in the sequence
3. Converts each word to its vector representation
4. Updates the hidden state using the RNN equation
5. After all words are processed, computes output from the final hidden state

Forward Pass Purpose

The purpose of the forward pass is to calculate the loss and get the final output. No training happens during forward pass — it is only forward computation. The order is always: forward pass → loss calculation → backward pass (gradient computation) → weight update.

3.5 Key Observations from Many-to-One

- In many-to-one RNN, output is generated only at the **final time step**
- Only a **single loss value** is computed
- The final hidden state contains **cumulative information** from all time steps
- If sequence length increases, the number of update steps increases, but **no padding is required** — that is a benefit of the recurrence relation
- The last step contains a **compressed memory** of all previous states (H1, H2, H3, H4), so all past information is involved in predicting the final output

4 Many-to-Many RNN: Practical Implementation

Many-to-Many RNN

Instead of waiting until the full sequence ends, an output is produced at **every time step**:

$$h_t = \tanh(W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1} + b_h)$$

$$y_t = W_{hy} \cdot h_t + b_y$$

The flow becomes:

$$x_1 \to h_1 \to y_1, \quad x_2 \to h_2 \to y_2, \quad x_3 \to h_3 \to y_3, \quad \dots$$

4.1 Toy Example: Next-Character Prediction

- Vocabulary: A, B, C (3 characters)
- Input sequence: A, B, C, A
- At each step, predict the next symbol pattern
- This is not a full language model — the goal is to understand structure

Many-to-Many Forward Pass

```python
import numpy as np
np.random.seed(7)
np.set_printoptions(precision=3, suppress=True)

char_to_vec = {
    "A": np.array([[1.0], [0.0], [0.0]]),
    "B": np.array([[0.0], [1.0], [0.0]]),
    "C": np.array([[0.0], [0.0], [1.0]])
}

idx_to_char = {0: "A", 1: "B", 2: "C"}

sequence = ["A", "B", "C", "A"]

input_size = 3
hidden_size = 4
output_size = 3

W_xh = np.random.randn(hidden_size, input_size) * 0.4
W_hh = np.random.randn(hidden_size, hidden_size) * 0.4
W_hy = np.random.randn(output_size, hidden_size) * 0.4

b_h = np.zeros((hidden_size, 1))
b_y = np.zeros((output_size, 1))

def softmax(z):
    exp_z = np.exp(z - np.max(z))
    return exp_z / np.sum(exp_z)

h = np.zeros((hidden_size, 1))

for t, token in enumerate(sequence, start=1):
    x_t = char_to_vec[token]
    h = np.tanh(W_xh @ x_t + W_hh @ h + b_h)
    y_t = W_hy @ h + b_y
    p_t = softmax(y_t)
    predicted_idx = np.argmax(p_t)
    predicted_char = idx_to_char[predicted_idx]
    print(f"Step {t}: Input={token}, Predicted={predicted_char}")
```

Listing 4: Character-level many-to-many RNN

4.2 Softmax Function

For many-to-many with multiple output classes, the softmax function converts raw output scores into probabilities:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Softmax is an extension of sigmoid. When there are only two classes, softmax reduces to sigmoid. For multi-class classification (more than two labels), softmax is used. For binary classification, sigmoid is sufficient.

4.3 Key Observations from Many-to-Many

- Hidden state keeps evolving through time
- Output is generated at **every** time step
- Later outputs depend on earlier sequence information
- This setup is more expressive than many-to-one for sequence generation and labeling
- The same token at different positions can produce **different outputs** because the hidden state carries past context
- After every time step, loss is calculated, so weights are updated based on losses at every step
- The choice between sigmoid and softmax depends on binary vs. multi-class classification, **not** on many-to-one vs. many-to-many

Why Repeated Tokens Produce Different Outputs

The hidden state carries past information (past context). So even if the same token is input at different sequential positions, the hidden state at that point is different, leading to different output predictions. For example, if the first input is A and the fourth input is also A, the predicted next token may differ because the hidden state at step 4 contains accumulated information from steps 1 through 3.

5 Comparison: Many-to-One vs Many-to-Many

Aspect	Many-to-One	Many-to-Many
Output timing	Final time step only	Every time step
Loss calculation	Single loss at the end	Loss at each time step
Use cases	Sentiment classification, sequence classification	Next token prediction, POS tagging, sequence labeling
Output representation	Final hidden state $\rightarrow$ output	Each hidden state $\rightarrow$ output
Training difference	Backprop from final output	Backprop from each step (BPTT at each step)
Many-to-One	Last step carries compressed memory of all previous states	Each step produces output while carrying memory forward

6 Training RNN from Scratch

Without training, predictions are random because weights are random and no learning has happened.

6.1 Training Components

To train an RNN, four components are needed:

1. **Loss Function** — measures how far the prediction is from the actual label
2. **Forward Pass** — compute the output
3. **Backward Pass** — compute the gradients
4. **Weight Update** — adjust weights using gradient descent

6.2 Training Flow

For each sequence:

Step 1 — Forward pass through time:

$$h_t = \tanh(W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1} + b_h)$$

Step 2 — Final prediction:

$$y = W_{hy} \cdot h_T + b_y$$

Step 3 — Compute loss (mean squared error):

$$L = (y - y_{\text{true}})^2$$

Step 4 — Update weights (gradient descent):

$$W = W - \eta \cdot \nabla W$$

where η is the learning rate.

6.3 Training Implementation (NumPy)

Training RNN from Scratch

```
import numpy as np
np.random.seed(1)

word_to_vec = {
    "good": np.array([[1.0],[0.0]]),
    "bad":  np.array([[0.0],[1.0]])
}

dataset = [
    (["good","good","good"], 1),
    (["bad","bad","bad"], 0),
    (["good","bad","good"], 1),
    (["bad","good","bad"], 0)
]

input_size = 2
hidden_size = 4
output_size = 1

W_xh = np.random.randn(hidden_size, input_size) * 0.5
W_hh = np.random.randn(hidden_size, hidden_size) * 0.5
W_hy = np.random.randn(output_size, hidden_size) * 0.5

b_h = np.zeros((hidden_size, 1))
b_y = np.zeros((output_size, 1))

learning_rate = 0.01
epochs = 50

for epoch in range(epochs):
    total_loss = 0
    for sequence, label in dataset:
        # Forward pass
        h = np.zeros((hidden_size, 1))
```

```
        states = []
        for word in sequence:
            x = word_to_vec[word]
            h = np.tanh(W_xh @ x + W_hh @ h + b_h)
            states.append(h)

        y = W_hy @ h + b_y

        # Loss (MSE)
        loss = (y - label)**2
        total_loss += loss

        # Gradient approximation
        dy = 2 * (y - label)
        dW_hy = dy @ states[-1].T

        # Backprop through last state (simplified)
        dh = W_hy.T @ dy
        dtanh = (1 - states[-1]**2) * dh
        dW_xh = dtanh @ word_to_vec[sequence[-1]].T

        # Weight update
        W_hy -= learning_rate * dW_hy
        W_xh -= learning_rate * dW_xh

    print(f"Epoch {epoch+1}, Loss: {total_loss.flatten()[0]:.4f}")
```

Listing 5: Training loop with simplified backpropagation

Simplified Backpropagation

In this implementation, backpropagation is applied only through the last hidden state for simplicity. In a real RNN, **Backpropagation Through Time (BPTT)** should go through every time step. This simplified version works because the last hidden state carries cumulative information from all previous steps (W_{hy}^T connects the output gradient back to the hidden state).

In actual implementation, BPTT unrolls the entire sequence and propagates gradients through all hidden states: $h_T \to h_{T-1} \to \ldots \to h_1$.

6.4 What to Observe During Training

- Loss reduces gradually over epochs
- Predictions improve as the model learns sequence patterns
- The RNN starts learning which sequences map to which labels
- Loss decreasing indicates the model is learning in the right direction

6.5 Training Concepts

- **Why compute loss?** To update the weights and improve accuracy
- **What does decreasing loss indicate?** The model is getting closer to the correct output; learning is going in the right direction
- **Why update weights?** To converge to better predictions
- **Why train over multiple epochs?** One pass through the data is rarely enough; repeated exposure helps the model refine its weights
- **What does learning rate control?** The speed of convergence — how large each weight update step is

7 Training Real RNN Using PyTorch

PyTorch provides efficient RNN implementation with built-in backpropagation, GPU support, and stable training. Instead of manually writing hidden loops, gradients, and updates, the model is defined using `nn.RNN`.

7.1 Many-to-One with PyTorch

PyTorch RNN Implementation

```python
import torch
import torch.nn as nn
import torch.optim as optim

torch.manual_seed(0)

word_to_idx = {"good": 0, "bad": 1}

dataset = [
    (["good","good","good"], 1),
    (["bad","bad","bad"], 0),
    (["good","bad","good"], 1),
    (["bad","good","bad"], 0)
]

def encode(sequence):
    return torch.tensor([word_to_idx[w] for w in sequence])

X = [encode(seq) for seq, _ in dataset]
y = torch.tensor([label for _, label in dataset]).float().unsqueeze(1)

class SimpleRNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.embedding = nn.Embedding(2, 4)
        self.rnn = nn.RNN(4, 8, batch_first=True)
        self.fc = nn.Linear(8, 1)

    def forward(self, x):
        x = self.embedding(x)
        output, hidden = self.rnn(x)
        final_hidden = hidden.squeeze(0)
        y = self.fc(final_hidden)
        return y

model = SimpleRNN()
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)

epochs = 100
for epoch in range(epochs):
    total_loss = 0
    for i in range(len(X)):
        optimizer.zero_grad()
        output = model(X[i].unsqueeze(0))
        loss = criterion(output, y[i].unsqueeze(0))
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    if epoch % 10 == 0:
```

```
51          print(f"Epoch {epoch}, Loss {total_loss:.4f}")
```

Listing 6: Real RNN training using PyTorch nn.RNN

7.2 PyTorch Model Architecture Breakdown

Layer	Purpose	Shape
`nn.Embedding(2, 4)`	Converts word indices to 4-dim vectors	vocab → embedding
`nn.RNN(4, 8, batch_first=True)`	RNN with input size 4, hidden size 8	embedding → hidden
`nn.Linear(8, 1)`	Maps final hidden state to output	hidden → output

7.3 Training Flow with PyTorch

1. Convert words to index tensors using `encode()`
2. Pass through embedding layer (words → vectors)
3. Pass through `nn.RNN` (vectors → hidden states)
4. Take the final hidden state
5. Pass through linear layer (hidden state → output)
6. Compute loss (MSE)
7. Backpropagate (`loss.backward()`)
8. Update weights (`optimizer.step()`)

Why Use nn.RNN Instead of Manual Implementation?

The manual NumPy implementation is useful for understanding the internal mechanics, but in practice, `nn.RNN` handles:

- Efficient computation with optimized kernels
- Automatic gradient computation via autograd
- GPU acceleration
- Stable numerical operations

The methodology of training remains the same: forward pass → loss calculation → backpropagation → weight update. This is true whether the model is a feedforward network, CNN, RNN, or even a transformer.

8 RNN Padding and Variable-Length Sequences

An important property of RNNs: **no padding is required** for variable-length sequences. This is a benefit of the recurrence relation.

If one sequence has 3 words and another has 4 words:

- The 3-word sequence simply has 3 time steps of hidden state updates
- The 4-word sequence has 4 time steps
- No padding to equalize lengths is necessary for the RNN itself

This is different from architectures that require parallel processing (e.g., transformers), where padding is used to create uniform batch sizes.

9 Backpropagation Through Time (BPTT)

BPTT in Many-to-Many

For many-to-many RNN, backpropagation through time works as follows:

- At each time step, output y_t is computed and loss L_t is calculated
- Weights are shared across all time steps
- The total gradient is the sum of gradients from all time steps:

$$\frac{dL}{dW} = \sum_{t=1}^{T} \frac{dL_t}{dW}$$

- Gradients flow backward through the hidden states: $h_T \to h_{T-1} \to \ldots \to h_1$

For many-to-one, the loss is calculated only at the final step, but gradients still propagate back through all hidden states during BPTT. The simplified version (backpropagating only through the last state) works for toy examples because the last state carries cumulative information.

10 Summary

- **Sequential data** requires models that process information in order; RNNs achieve this through a hidden state that acts as running memory
- The **basic RNN update**: $h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$
- **Many-to-One**: entire sequence $\to$ single output (e.g., sentiment classification); loss computed only at the final step
- **Many-to-Many**: output at every time step (e.g., next token prediction); loss computed at each step
- The same input token at different positions can produce different outputs because the hidden state carries past context
- **Training flow**: forward pass $\to$ loss computation $\to$ backpropagation $\to$ weight update (same methodology for all neural networks)
- **No padding required** for variable-length sequences in RNN due to the recurrence relation
- PyTorch `nn.RNN` provides efficient implementation with automatic gradient computation, replacing manual NumPy implementations
- Softmax is used for multi-class classification; sigmoid for binary classification — this choice depends on the number of classes, not on many-to-one vs. many-to-many
- Weights must be initialized randomly (not zero) to avoid vanishing gradients in the first pass